

APPENDIX

AskATT System1 API

A JEV-shaped Noul and Choice service implemented with GLiClass Modern Large v3

INDEPENDENT IMPLEMENTATION	LOCKED TEST	SHARED PASS
API compatibility, not a reproduction of JEV	150 transcripts 4,050 decisions	27 labels in one GLiClass pass

Executive result

The service accepts one shared state plus named typed questions at `POST /v1/systemone`. It compiles Noul and Choice questions into GLiClass labels and returns the familiar structured answer envelope. On the held-out test, Noul and conventional binary Choice each reached 94.10% accuracy, 96.07% recall, and 89.57% F1.

This is an interface experiment, not a claim that GLiClass duplicates JEV's weights, calibration, confidence, limits, training method, or proprietary architecture.

The current TypeSafe documentation states that JEV ingests the state once and evaluates questions in parallel. AskATT reaches a similar external shared-state pattern using GLiClass's uni-encoder path; the mechanisms are not asserted to be equivalent.

Mark Austin | September 20, 2026

Two users, two operating decisions

DATA SCIENTIST / APPLICATION TEAM	ENTERPRISE PLATFORM / ASK AT&T
Optimize for learning speed. Start with an API or local CPU checkpoint, preserve one request schema, calibrate per-label thresholds, and measure the workload before reserving hardware.	Optimize a multi-tenant portfolio. Pool stable base load, retain hosted burst/fallback capacity, version models and policies, and operate to p95/p99 latency rather than theoretical saturation.
Bring evidence: label-level quality, state lengths, question counts, request rate, burstiness, privacy constraints, and latency SLO.	Provide isolation, auth, quotas, batching, autoscaling, observability, rollout controls, data-retention policy, and a funded reliability model.

When should a platform move off a usage-priced service?

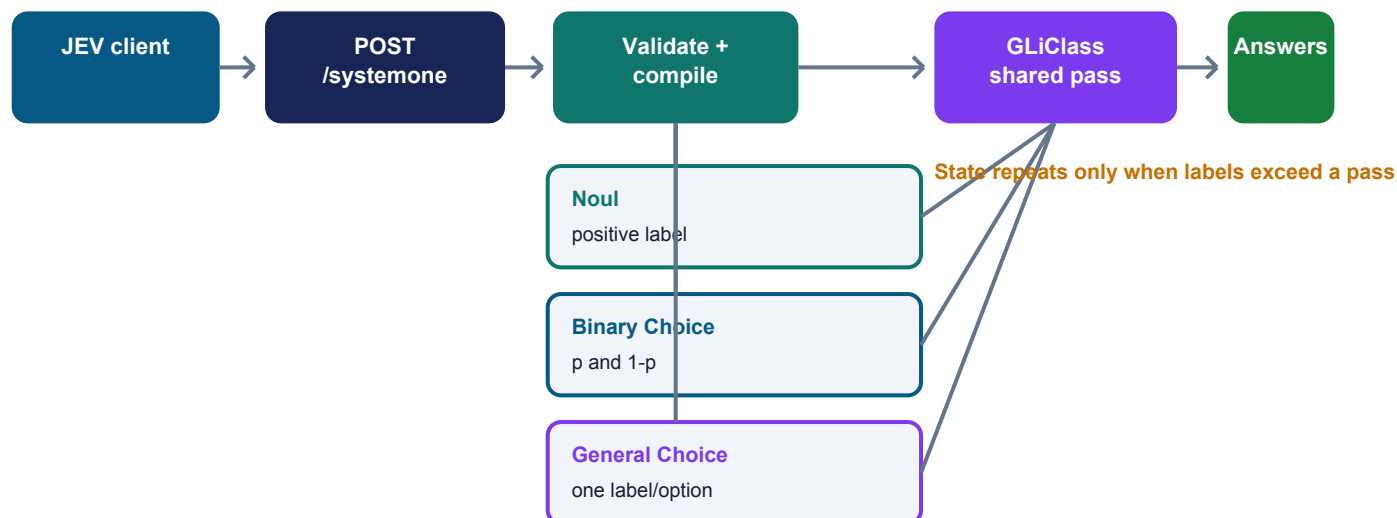
- Quality parity is demonstrated on a labeled production shadow set.
- Sustained demand - not peak demand - exceeds the all-in cost crossover.
- The shared pool can keep accelerators usefully occupied across tenants.
- Measured p95/p99 latency meets the service objective with capacity headroom.
- Reliability, security, privacy, operations, and failure recovery are funded.

Reference economics

At 6,000 state tokens and 25 questions, the report's H100 proxy estimates \$0.051 per 1,000 states for GLiClass at 100% paid utilization versus about \$0.352 for JEV Noul. The raw-inference crossover is 14.6% utilization. It is not a migration trigger: real self-hosting adds redundancy, orchestration, engineering, idle failover capacity, and support. A platform review at roughly 25-40% sustained utilization is a more prudent starting point, followed by an all-in measured business case.

Hybrid default: local shared pool for proven base load; hosted JEV or another managed service for bursts, fallback, long contexts, and labels that miss the local quality target.

How the compatibility layer works



Noul

The positive criterion - or the instructions when no positive criterion is supplied - becomes one GLiClass label. The returned score is the Noul probability.

Binary Choice

For present/absent, yes/no, or true/false options, the positive score is p and the other option is $1-p$. This makes Noul and binary Choice intentionally equivalent at a 0.50 selection threshold.

General Choice

Each option becomes a label. The adapter applies a within-question softmax over GLiClass score log-odds. Confidence is normalized distribution concentration; it is not TypeSafe's confidence calculation.

Pass count

Up to 64 compiled labels run in one shared-state pass by default. Additional label groups repeat the state. A multi-option Choice may therefore consume multiple compiled-label slots even though it is one question object.

Compatibility is deliberately bounded

Capability	TypeSafe JEV	AskATT implementation
Endpoint	POST /v1/systemone	Same path
State	String, object, or array	Same shapes; serialized to text
Noul / Choice	Both; Choice up to 255 options	Both; Choice up to 255 options
Score	Supported	Not implemented; HTTP 422
Context	64k total; state + longest question <=32k	8,192-token GLiClass input
Question cap	Token/rate limits govern	64 question objects by default
Confidence	JEV-defined	Normalized entropy proxy
Usage	JEV billable tokens	Serialized GLiClass input tokens
Auth	Bearer token	Optional bearer token
Model field	Selects alias/version	Required; local configured model answers

The service exposes extra diagnostic headers: inference milliseconds, forward-pass count, compiled-label count, and truncation status. These make state repetition and context loss observable to the caller.

Sources

TypeSafe API reference: <https://docs.typesafe.ai/api>

TypeSafe models and limits: <https://docs.typesafe.ai/models>

Accessed September 20, 2026.

Test results

Six deterministic contract tests and one opt-in real-checkpoint integration test passed. The full benchmark used the locked 150-transcript test split with 27 attributes per transcript: 4,050 binary decisions.

Measure	Noul	Binary Choice
Accuracy	94.10%	94.10%
Precision	83.89%	83.89%
Recall	96.07%	96.07%
F1	89.57%	89.57%
Exact transcript match	16.67%	16.67%
TP / FP / FN / TN	1,026 / 197 / 42 / 2,785	Same
Input tokens	78,219	78,219
Forward passes/state	1.0	1.0
Truncated rows	0	0
Local CPU wall time	78.62 s	83.10 s
Local CPU states/s	1.91	1.81

The Noul and binary Choice predictions were identical. Their small CPU timing difference is run-to-run overhead rather than different model work.

H100 capacity estimate

Using the report's 170,331 processed-token/s H100 proxy and \$5/GPU-hour assumption, the benchmark's 521.46 serialized tokens imply approximately 3.06 ms of saturated GPU resource time per state and \$0.00425 per 1,000 states at 100% utilization. This is not measured end-to-end request latency.

Calling the API

```
curl http://127.0.0.1:8000/v1/systemone \
+ -H 'Content-Type: application/json' \
+ -d '{
  "model": "jev-latest",
  "state": "The caller disputes an unfamiliar fee.",
  "questions": {
    "unknown_charge": {
      "type": "noul",
      "instructions": "Is an unknown charge present?",
      "criteria": {"true": "Unknown charges on the bill"}
    },
    "department": {
      "type": "choice",
      "instructions": "Route the issue",
      "criteria": {
        "billing": "Billing, charges, and payments",
        "technical": "Internet or device failure",
        "sales": "New purchases or upgrades"
      }
    }
  }
}'
```

Representative response

```
{
  "model": "askatt-gliclass-modern-large-v3.0",
  "answers": {
    "unknown_charge": {"type": "noul", "noul": 0.7835},
    "department": {
      "type": "choice", "choice": "billing",
      "probabilities": {"billing": 0.9890, "technical": 0.0067,
        "sales": 0.0043},
      "confidence": 0.932
    }
  },
  "usage": {"input_tokens": 61, "output_tokens": 0}
}
```

The response uses JEV field names while truthfully identifying the configured AskATT model. The request's `model` is required for schema compatibility but does not dynamically load a checkpoint.

Cost, utilization, and latency

```
passes = ceil(compiled_labels / labels_per_pass)
processed_tokens ~= passes * state_tokens + serialized_label_tokens
resource_seconds/state = processed_tokens / saturated_tokens_per_second
cost/1,000 states = 1,000 * resource_seconds/state * GPU_dollars/hour
/ (3,600 * paid_utilization)

JEV billable_tokens ~= state_tokens + sum(question_tokens) + overhead
JEV cost/1,000 = 1,000 * billable_tokens * dollars/input_token

end_to_end_latency = queueing + preprocessing + model_service
+ postprocessing + network
```

The formula makes the platform trade-off visible. At low paid utilization, idle GPU hours dominate. As utilization approaches saturation, cost per state falls but queueing grows sharply. Offline jobs can run near saturation; interactive services need headroom and should be governed by measured percentile latency.

Reproduce

```
python3 -m venv --system-site-packages .venv
.venv/bin/pip install -r askatt_system1_api/requirements.txt
export ASKATT_GLICLASS_MODEL=models/gliclass-modern-large-v3.0
export ASKATT_DEVICE=cpu
.venv/bin/python -m askatt_system1_api

.venv/bin/python -m pytest -q tests/test_askatt_system1_api.py
ASKATT_RUN_MODEL_TESTS=1 .venv/bin/python -m pytest -q tests/test_askatt_system1_model.py
.venv/bin/python scripts/benchmark_askatt_system1_api.py
```

Production checklist

- Version prompts, criteria, thresholds, and model checkpoints together.
- Benchmark CUDA with representative concurrency, state lengths, and question counts.
- Add dynamic batching, back-pressure, tenant-scoped auth, quotas, and audit logs.
- Canary against pinned JEV versions and a labeled production shadow set.
- Route proven base load locally; retain a hosted fallback until reliability and quality are established.